PROJECT REPORT

High-Level Design Document

RAG-Based Customer Support Assistant
with LangGraph & Human-in-the-Loop

Name: Om Dongare

Intern ID: IN22603402

Date: 24/04/2026

Document Type: High-Level Design (HLD)

Contents

1 SystemOverview 2
2 ArchitectureDiagram 4
3 ComponentDescription 6
4 DataFlow 8
5 TechnologyChoices 9
6 ScalabilityConsiderations 10

System Overview

1.1 Problem Definition

Traditional customer support systems face several critical challenges:

- High Volume: Support teams are overwhelmed with repetitive queries that could be automated.
- Inconsistency: Different agents provide varying quality of answers to the same questions.
- Latency: Customers wait in queues for simple answers that exist in documentation.
- Knowledge Gaps: New agents lack institutional knowledge, leading to incorrect or incomplete responses.
- Scalability: Hiring and training human agents is expensive and slow.

The fundamental problem is:
How can we deliver instant, accurate, and contextually relevant customer support at scale while maintaining the ability to escalate complex or sensitive cases to human agents?

1.2 Scope of the System

This system is a Retrieval-Augmented Generation (RAG) customer support assis- tant. Table 1.1 summarises what is in and out of scope.

Table 1.1: System Scope

In Scope	Out of Scope Multi-
PDF-based knowledge base inges- tion	language support
Semantic search with vector em- b eddings	Real-time knowledge base updates
LLM-powered answer generation	Payment processing or account modifi- cations
Intent-based query classification & routing	Integration with external CRM systems
Human-in-the-Loop (HITL) esca- lation	Multi-modal input (images, voice)
Web-based (Streamlit) chat interface	Mobile application
Conversation history within a ses- sion	Cross-session memory persistence

1.3 KeyDesignPrinciples

1. Accuracy over Speed: The system prioritises correct answers over fast but wrong ones. When uncertain, it escalates.
2. Transparency: Every response shows the source documents used, building user trust.
3. Graceful Degradation: When components fail (LLM errors, empty retrieval), the system falls back to human escalation rather than crashing.
4. Modularity: Each component is independently replaceable (swap ChromaDB for Pinecone, swap Llama for GPT-4, etc.).

Architecture Diagram

2.1 System Architecture

The system is structured into five interconnected layers as shown in Figure 2.1: the User Interface Layer, Workflow Orchestration Layer, AI Processing Layer, Data Storage Layer, and HITL System Layer.

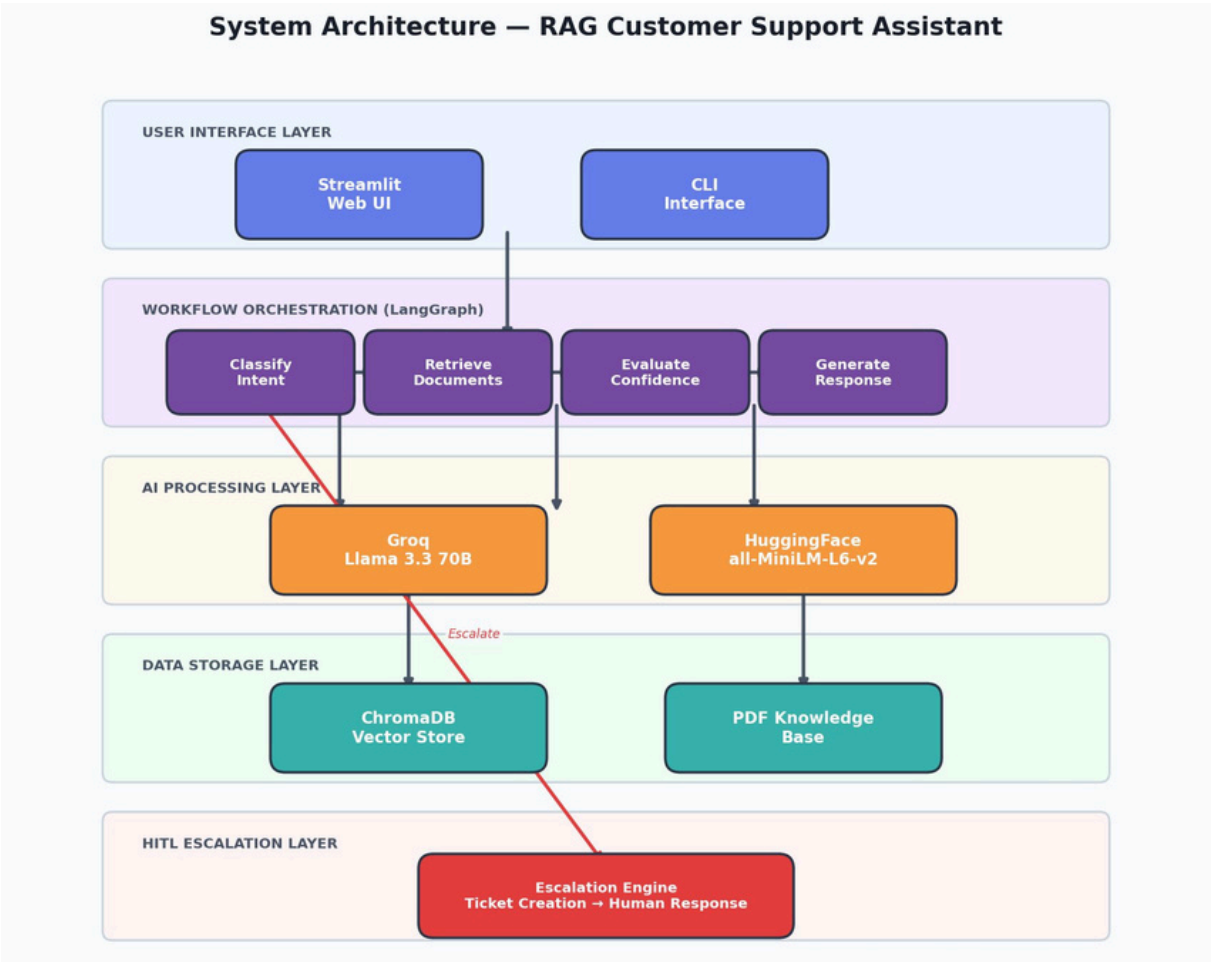

Figure 2.1: System Architecture — RAG Customer Support Assistant

2.2 LangGraph Workflow

Figure 2.2 shows the LangGraph StateGraph with conditional routing. The workflow consists of five nodes connected by conditional edges that route queries based on intent and retrieval confidence.

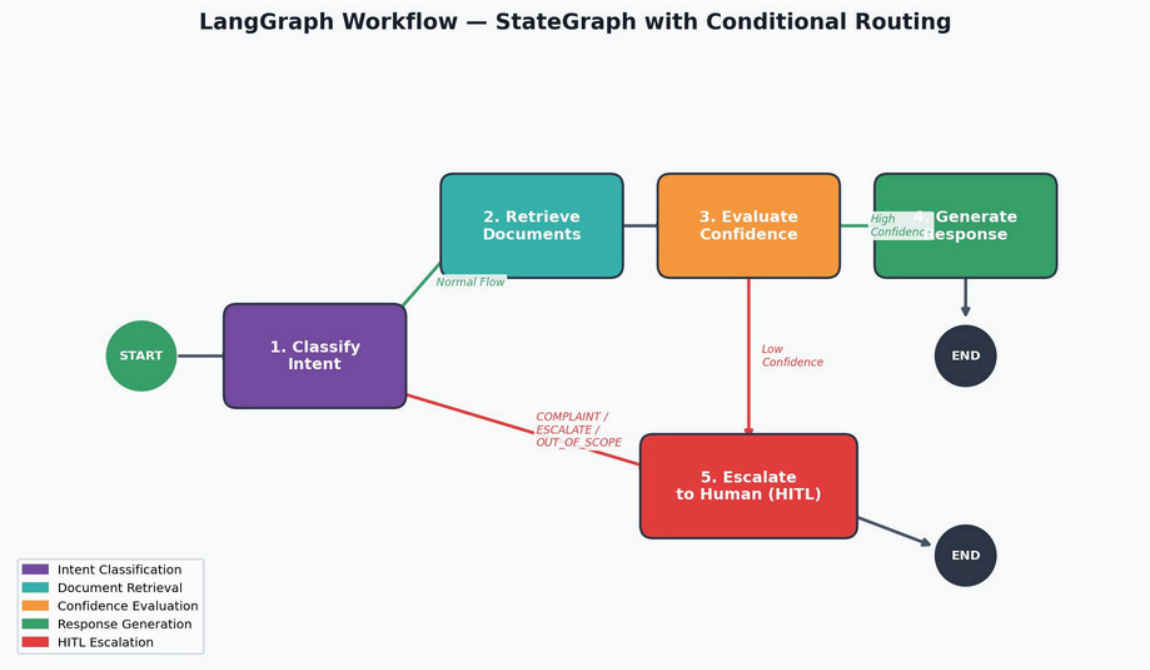

Figure 2.2: LangGraph Workflow — StateGraph with Conditional Routing

Component Description

3.1 DocumentLoader

- Purpose: Extracts text from PDF documents and converts them into structured document objects.
- Technology: PyPDFLoader (LangChain Community)
- Input: PDF file path
- Output: List of Document objects with page content and metadata
- Design Decision: Chosen for its simplicity, speed, and reliability with standard PDF formats. Processes page-by-page, preserving document structure.

3.2 Chunking Strategy

- Technology: RecursiveCharacterTextSplitter
- Chunk Size: 500 characters (optimal balance between context and precision) Chunk
- Overlap: 100 characters (20% overlap to prevent context loss at boundaries)
- Rationale: Recursive splitting preserves semantic meaning better than fixed-size splitting by trying natural text boundaries first.

3.3 Embedding Model

- Technology: HuggingFace Embeddings (all-MiniLM-L6-v2) — runs locally
- Output Dimension: 384-dimensional vectors
- Benefits: Zero API cost, no rate limits, works offline, fast inference with small footprint (~80 MB)

3.4 VectorStore

- Technology: ChromaDB (persistent mode)
- Storage: Local filesystem (./chroma db/)
- Search Method: Cosine similarity
- Rationale: Lightweight, self-contained, persistent, open-source, with first-class LangChain integration.

3.5 Retriever

- Configuration: Top-K = 3
- Features: Similarity search with relevance scores (0.0–1.0), confidence evaluation for HITL routing, context formatting with source attribution
- Rationale: K=3 provides sufficient context without overwhelming the LLM prompt.

3.6 LLM

- Technology: Groq (Llama 3.3 70B Versatile)
- Temperature: 0.3 (low for factual, consistent responses)
- Rationale: Generous free tier (14,400 requests/day), ultra-fast inference via Groq's LPU hardware, excellent reasoning and factual accuracy.

3.7 GraphWorkflowEngine

- Technology: LangGraph StateGraph
- No des: 5 processing nodes (classify, retrieve, evaluate, generate, escalate)
- Edges: 2 conditional edges + 2 direct edges
- State: TypedDict with 12 fields tracking query lifecycle

3.8 HITLModule

- Purpose: Handles transition from automated to human-assisted support
- Features: Escalation trigger evaluation, context-rich ticket creation, human response integration, user-facing escalation messages
- Production Path: Would connect to a ticketing system (e.g., Zendesk, Freshdesk)

Data Flow

4.1 RAG Pipeline — PDF to Response

Figure 4.1 illustrates the complete data flow: the document ingestion pipeline (top) con- verts PDFs to vector embeddings stored in ChromaDB, while the query processing pipeline (bottom) routes user queries through intent classification, retrieval, confidence evaluation, and either LLM response generation or HITL escalation.

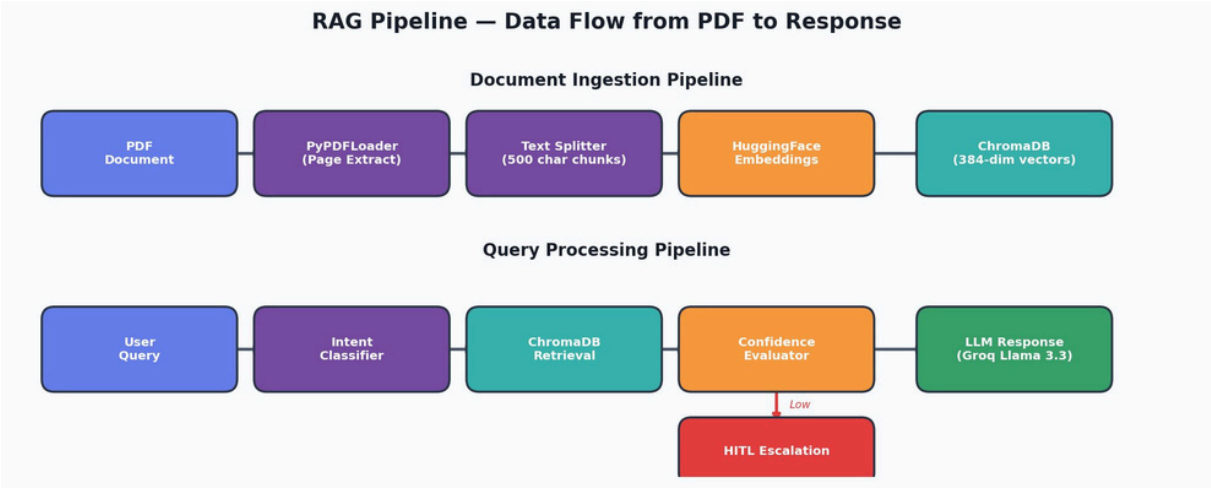

Figure 4.1: RAG Pipeline — Data Flow from PDF to Response

4.1.1 DocumentIngestionPipeline Data

Transformations:

1. PDF → Pages: Binary PDF → list of text strings (one per page)
2. Pages → Chunks: Long text → 500-char segments with 100-char overlap
3. Chunks → Embeddings: Text strings → 384-dimensional float vectors
4. Embeddings → ChromaDB: Vectors stored with metadata (source, page, chunk index)

4.1.2 QueryLifecycle

1. Intent Classification (LLM): Output — intent category + escalation flag
2. Document Retrieval (ChromaDB): Output — top-3 docs with relevance scores
3. Confidence Evaluation: Output — confidence level + escalation flag
4. Response Generation (LLM): Input — query + context; Output — answer string
5. HITL Escalation (if triggered): Output — ticket + user message

Technology Choices

5.1 WhyChromaDB?

Table 5.1: ChromaDB vs Alternatives

Criterion	ChromaDB	Pinecone/Weaviate
Setup	Zeroconfig,localfile	Requires cloud account or Do cker
Cost	Free,open-source	Freetierslimited,thenpaid
Persistence	Built-infilepersistence	Varies
LangChain	First-classsupport	Supportedbutmoresetup
Pro duction	Good for small-medium scale	Better for large scale

5.2 WhyLangGraph?

Table 5.2: LangGraph vs Alternatives

Criterion Conditional	LangGraph	LCEL/Custom
Rout- ing	Nativesupport	Requirescustomlogic
State Management	Built-inTypedDictstate	Manualimplementation
HITL Support	interrupt()function	Nobuilt-insupport
Visualization	Graphvisualization	None
Debugging	Clearnode-by-node	Opaquechainexecution

5.3 LLM Choice: Groq (Llama 3.3 70B Versatile)

- Free tier: 14,400 req/day, no credit card required
- Speed: Ultra-fast inference via Groq's custom LPU hardware
- Quality: Llama 3.3 70B offers excellent reasoning and factual accuracy
- Temperature 0.3: Prioritises consistent, factual responses over creative ones
- Alternatives considered: OpenAI GPT-4o (better quality but costly), Google Gemini Flash (similar tier but slower)

Scalability Considerations

6.1 HandlingLargeDocuments

Table 6.1: Large Document Scalability

Challenge	Current Approach	Scaling Strategy
Large PDFs (100+ pages)	Single-threadedprocessing	Batch processing with multiprocessing
MultiplePDFs	Sequentialingestion	Parallelingestionpipeline
Chunkvolume(10,000+)	ChromaDBlocal	Migrate to Pinecone/Weaviate cloud
Storagegrowth	Localfilesystem	Cloudobjectstorage(S3)

6.2 Latency Concerns

Table 6.2: Latency and Optimisation

Op eration	Current Latency	Optimisation
Intent Classification	~ 1–2s	Cache common intents; keyword pre-filter
Vector Search	~ 50–100ms	Quantised embeddings; ANN
LLM Generation	~ 2–4s	Streaming; model distillation
Total End-to-End	~ 4–8s	Pipeline parallelisation

6.3 Future Architecture Considerations

For production deployment at scale:

1. Microservices: Split into ingestion, retrieval, and generation services
2. Message Queue: RabbitMQ/Kafka for async processing between services
3. Caching Layer: Redis cache for frequent queries and intent classifications
4. Monitoring: LangSmith tracing for production observability
5. CDN: Serve the web UI from a CDN for global access
6. Auto-scaling: Kubernetes-based deployment with horizontal pod autoscaling